

Qontint Intelligence Engine - Implementation Report

Overview of the Deployed and Working Features

1. Architecture Overview

The Qontint web application is fully built and deployed on the Render platform. It features a FastAPI backend providing high-performance API endpoints and a Vite/React frontend. The core infrastructure relies on a highly scalable, free-stack approach utilizing PostgreSQL for relational data, Neo4j for graph-based taxonomy mapping, and Redis for caching and task queues via Celery.

2. AI-Driven NLP & Analytics Pipeline

The backend is powered by a robust, deterministic NLP scoring architecture designed for B2B Fintech Query Intelligence. Key working modules include:

- M1 SERP Collector: Real-time search engine results collection via DuckDuckGo and httpx.
- M2 Entity Extractor: SpaCy-based extraction and SVO-triple relationship mapping.
- M3 Graph Builder: Neo4j PageRank algorithm for graph taxonomy generation.
- M4/M5 Novelty & Authority: Real-time scoring using graph density and baseline metrics.
- M6 Ranking Predictor: Machine learning-based SEO position estimation using GradientBoosting.
- M9 Content Generator: Integration with local/remote LLMs (Ollama / Gemini) to dynamically generate content that fulfills sub-20-second latency targets.

3. Frontend UI/UX & Visualization

The frontend has undergone a cinematic UI overhaul to provide a premium enterprise SaaS dashboard experience:

- Visuals: GPU-accelerated ambient visuals and interactive physics using Three.js and React Three Fiber (@react-three/drei).
- UI Components: Modular glassmorphism design with responsive TailwindCSS layouts and smooth animations via Framer Motion & GSAP.
- Functionality: Real-time dynamic graph visualization pages, keyword taxonomy explorations, and data charting using Recharts.

Qontint Intelligence Engine - Implementation Report

Overview of the Deployed and Working Features

4. Deployment Infrastructure

The application has been successfully migrated and orchestrated on the Render cloud platform. Key infrastructure implementations include:

- Persistent Storage: Configured properly to retain SQLite states and essential caches across deployments.
- Optimization: Bottlenecks in the intelligence pipeline were addressed via batch processing and optimizing database interactions, maintaining a sub-20-second analysis latency.
- Stability: Unhandled backend exceptions during content generation (Gemini integration) have been identified and resolved, ensuring smooth API communication.